Phase 1: The FHE Dream and The Bootstrapping Blueprint

Fully Homomorphic Encryption over the Integers

Paper Presentation

April 13, 2026

Outline: Phase 1

1. Motivation & The Integer Paradigm
2. Formalizing Fully Homomorphic Encryption
3. The Bootstrapping Blueprint
4. References

Motivation: The Cloud Computing Conundrum

Motivation

We possess robust cryptographic solutions for data **in transit** *(TLS/SSL) and data* **at rest** *(AES). However, computing on that data traditionally requires decryption, creating a massive vulnerability when outsourcing to untrusted cloud servers.*

Motivation: The Cloud Computing Conundrum

Motivation

We possess robust cryptographic solutions for data **in transit** *(TLS/SSL) and data* **at rest** *(AES). However, computing on that data traditionally requires decryption, creating a massive vulnerability when outsourcing to untrusted cloud servers.*

The Fully Homomorphic Encryption (FHE) Goal: Allow an untrusted party to take encrypted inputs, perform arbitrary, unbounded computations on them, and return an encrypted result.

Motivation: The Cloud Computing Conundrum

Motivation

We possess robust cryptographic solutions for data **in transit** *(TLS/SSL) and data* **at rest** *(AES). However, computing on that data traditionally requires decryption, creating a massive vulnerability when outsourcing to untrusted cloud servers.*

The Fully Homomorphic Encryption (FHE) Goal: Allow an untrusted party to take encrypted inputs, perform arbitrary, unbounded computations on them, and return an encrypted result.

When decrypted, this result must match exactly what the data owner would have obtained by computing on the plaintext locally.

The Quest for the Simplest Secure Scheme

"What is the simplest encryption scheme for which one can hope to achieve security?"

The Quest for the Simplest Secure Scheme

"What is the simplest encryption scheme for which one can hope to achieve security?"

The Caesar Cipher Analogy:

- Shift 'A' by 3 and 'B' by 3. You can trivially add those shifts together.
- It is perfectly (and ironically) homomorphic under addition.
- **Fatal Flaw:** It is completely insecure.

The Quest for the Simplest Secure Scheme

"What is the simplest encryption scheme for which one can hope to achieve security?"

The Caesar Cipher Analogy:

- Shift 'A' by 3 and 'B' by 3. You can trivially add those shifts together.
- It is perfectly (and ironically) homomorphic under addition.
- **Fatal Flaw:** It is completely insecure.

The Paradigm Shift (2009 vs. 2010):

- Craig Gentry's breakthrough 2009 FHE scheme utilized ideal lattices over polynomial rings—conceptually and mathematically heavy machinery.

The Quest for the Simplest Secure Scheme

"What is the simplest encryption scheme for which one can hope to achieve security?"

The Caesar Cipher Analogy:

- Shift 'A' by 3 and 'B' by 3. You can trivially add those shifts together.
- It is perfectly (and ironically) homomorphic under addition.
- **Fatal Flaw:** It is completely insecure.

The Paradigm Shift (2009 vs. 2010):

- Craig Gentry's breakthrough 2009 FHE scheme utilized ideal lattices over polynomial rings—conceptually and mathematically heavy machinery.
- **This Paper:** Asks if we can achieve the exact same FHE blueprint using only elementary addition and multiplication over the integers.

Formalizing the FHE Dream

To prove a scheme works, we must rigorously define it. A homomorphic public key encryption scheme $\mathcal{E}$ consists of four Probabilistic Polynomial-Time (PPT) algorithms:

Formalizing the FHE Dream

To prove a scheme works, we must rigorously define it. A homomorphic public key encryption scheme $\mathcal{E}$ consists of four Probabilistic Polynomial-Time (PPT) algorithms:

$$\mathcal{E} = (\text{KeyGen}, \text{Encrypt}, \text{Decrypt}, \textbf{Evaluate})$$

The Evaluate Algorithm: Takes as input a public key pk, a boolean circuit C, and a tuple of ciphertexts $\vec{c} = \langle c_1, ..., c_t \rangle$ (corresponding to the t input bits of C), and outputs a new evaluated ciphertext c_{out}.

Correctness and Homomorphism

Definition (Correct Homomorphic Decryption)

The scheme $\mathcal{E}$ is correct for a given t-input circuit C if, for any key-pair $(sk, pk) \leftarrow \text{KeyGen}(\lambda)$, any plaintexts $m_1, ..., m_t \in \{0, 1\}$, and any ciphertexts $c_i \leftarrow \text{Encrypt}(pk, m_i)$, the following holds with overwhelming probability:

$$\text{Decrypt}(sk, \text{Evaluate}(pk, C, \vec{c})) = C(m_1, ..., m_t)$$

Correctness and Homomorphism

Definition (Correct Homomorphic Decryption)

The scheme $\mathcal{E}$ is correct for a given t-input circuit C if, for any key-pair $(sk, pk) \leftarrow \text{KeyGen}(\lambda)$, any plaintexts $m_1, ..., m_t \in \{0, 1\}$, and any ciphertexts $c_i \leftarrow \text{Encrypt}(pk, m_i)$, the following holds with overwhelming probability:

$$\text{Decrypt}(sk, \text{Evaluate}(pk, C, \vec{c})) = C(m_1, ..., m_t)$$

Definition (Fully Homomorphic Encryption)

The scheme $\mathcal{E}$ is homomorphic for a class $\mathcal{C}$ of circuits if it is correct for all circuits $C \in \mathcal{C}$. The scheme $\mathcal{E}$ is **fully homomorphic** if it is correct for *all* boolean circuits (i.e., circuits of arbitrary depth and size).

The "Trivial" Solution and The Compactness Rule

Mathematical Loophole: Can we satisfy FHE right now?

The "Trivial" Solution and The Compactness Rule

Mathematical Loophole: Can we satisfy FHE right now?
Yes. An untrusted server could just take the circuit C, concatenate it to the raw input ciphertexts, and send it back. The user decrypts the inputs, reads C, and computes it locally.

The "Trivial" Solution and The Compactness Rule

Mathematical Loophole: Can we satisfy FHE right now?
Yes. An untrusted server could just take the circuit C, concatenate it to the raw input ciphertexts, and send it back. The user decrypts the inputs, reads C, and computes it locally.

The Fix: Compactness. To prevent the user from doing the computational work, the server must compress the answer.

The "Trivial" Solution and The Compactness Rule

Mathematical Loophole: Can we satisfy FHE right now?
Yes. An untrusted server could just take the circuit C, concatenate it to the raw input ciphertexts, and send it back. The user decrypts the inputs, reads C, and computes it locally.

The Fix: Compactness. To prevent the user from doing the computational work, the server must compress the answer.

Definition (Compactness)

A homomorphic scheme $\mathcal{E}$ is compact if there exists a fixed polynomial bound $b(\lambda)$ such that the size of the ciphertext output by Evaluate$(pk, C, \vec{c})$ is strictly bounded by $b(\lambda)$ bits, **independently of the size or depth of** C.

The Crisis of Noise

Intuition (The Noise Bottleneck)

In all known FHE schemes, ciphertexts contain "noise" to ensure security.

- *Homomorphic addition adds the noise.*
- *Homomorphic multiplication* **multiplies** *the noise.*

Because multiplication squares the noise, evaluating a deep circuit causes the noise to explode past the decryption boundary, permanently destroying the plaintext.

The Crisis of Noise

Intuition (The Noise Bottleneck)

In all known FHE schemes, ciphertexts contain "noise" to ensure security.

- *Homomorphic addition adds the noise.*
- *Homomorphic multiplication* **multiplies** *the noise.*

Because multiplication squares the noise, evaluating a deep circuit causes the noise to explode past the decryption boundary, permanently destroying the plaintext.

This limits us to a *Somewhat* Homomorphic Encryption (SHE) scheme. How do we compute infinitely if noise acts as a strict depth limit?

Defining Bootstrappability

The answer is Gentry's Bootstrapping Blueprint. First, we define a specific class of circuits.

Defining Bootstrappability

The answer is Gentry's Bootstrapping Blueprint. First, we define a specific class of circuits.

Definition (Augmented Decryption Circuits)

Let $\mathcal{E}$ be an encryption scheme where decryption is implemented by a boolean circuit. The set of augmented decryption circuits, $D_{\mathcal{E}}$, consists of two circuits:

1. Decrypt two ciphertexts and ADD the results.
2. Decrypt two ciphertexts and MULTIPLY the results.

Defining Bootstrappability

The answer is Gentry's Bootstrapping Blueprint. First, we define a specific class of circuits.

Definition (Augmented Decryption Circuits)

Let $\mathcal{E}$ be an encryption scheme where decryption is implemented by a boolean circuit. The set of augmented decryption circuits, $D_{\mathcal{E}}$, consists of two circuits:

1. Decrypt two ciphertexts and ADD the results.
2. Decrypt two ciphertexts and MULTIPLY the results.

Definition (Bootstrappable Encryption)

Let $\mathcal{C}_{\mathcal{E}}$ be the set of permitted circuits a scheme can evaluate before noise exceeds the bounds. A scheme is **bootstrappable** if it can evaluate its own augmented decryption circuits:

$$D_{\mathcal{E}} \subseteq \mathcal{C}_{\mathcal{E}}$$

Gentry's Bootstrapping Theorem

Theorem (Gentry's Transformation)

There is an efficient transformation that, given a description of a bootstrappable scheme $\mathcal{E}$, outputs a description of a **Fully Homomorphic** *encryption scheme capable of evaluating circuits of any depth.*

Gentry's Bootstrapping Theorem

Theorem (Gentry's Transformation)

There is an efficient transformation that, given a description of a bootstrappable scheme $\mathcal{E}$, outputs a description of a **Fully Homomorphic** *encryption scheme capable of evaluating circuits of any depth.*

Intuition (How Bootstrapping "Refreshes" Ciphertexts)

1. *Take a ciphertext c whose noise is dangerously high.*
2. *Encrypt c again under a new public key.*
3. *Provide the server with an encrypted secret key.*
4. *The server homomorphically evaluates the decryption algorithm on the inner ciphertext.*

Gentry's Bootstrapping Theorem

Theorem (Gentry's Transformation)

There is an efficient transformation that, given a description of a bootstrappable scheme $\mathcal{E}$, outputs a description of a **Fully Homomorphic** *encryption scheme capable of evaluating circuits of any depth.*

Intuition (How Bootstrapping "Refreshes" Ciphertexts)

1. *Take a ciphertext c whose noise is dangerously high.*
2. *Encrypt c again under a new public key.*
3. *Provide the server with an encrypted secret key.*
4. *The server homomorphically evaluates the decryption algorithm on the inner ciphertext.*

Result: *Because the decryption circuit is evaluated homomorphically, the output is a fresh ciphertext encrypting the exact same message, but with a fixed, smaller noise level determined solely by the depth of the decryption circuit itself!*

References

- Marten van Dijk, Craig Gentry, Shai Halevi, and Vinod Vaikuntanathan. *Fully Homomorphic Encryption over the Integers*. Eurocrypt, 2010.
- Craig Gentry. *Fully Homomorphic Encryption Using Ideal Lattices*. STOC, 2009.
- Craig Gentry. *A Fully Homomorphic Encryption Scheme*. PhD thesis, Stanford University, 2009.

Phase 2: The Integer Engine and The Noise Crisis

Fully Homomorphic Encryption over the Integers

Paper Presentation

April 13, 2026

Outline: Phase 2

1. The Symmetric & Public-Key SHE Schemes
2. Rigorous Proof of Correctness
3. The Multiplicative Bottleneck
4. References

Building the Engine: The Symmetric "Toy" Scheme

Before we look at the complex public-key version, we start with a beautifully simple symmetric-key foundation.

Building the Engine: The Symmetric "Toy" Scheme

Before we look at the complex public-key version, we start with a beautifully simple symmetric-key foundation.

Parameters: Let η be the bit-length of the secret key, and ρ be the bit-length of the noise.

Building the Engine: The Symmetric "Toy" Scheme

Before we look at the complex public-key version, we start with a beautifully simple symmetric-key foundation.

Parameters: Let η be the bit-length of the secret key, and ρ be the bit-length of the noise.

The Symmetric Algorithms

- **KeyGen:** The secret key p is a randomly chosen odd integer $p \in [2^{\eta-1}, 2^{\eta})$.
- **Encrypt(p, m):** To encrypt a bit $m \in \{0, 1\}$, choose a large random integer q and a small random noise integer r (where $|r| < 2^{\rho}$).

$$c = p \cdot q + 2r + m$$

- **Decrypt(p, c):** Output $m' = (c \pmod{p}) \pmod 2$.

Intuition: Why Symmetric Decryption Works

Let's break down the decryption equation: $m' = (c \pmod{p}) \pmod{2}$.

Intuition: Why Symmetric Decryption Works

Let's break down the decryption equation: $m' = (c \pmod{p}) \pmod{2}$.

1. Substitute c: $(p \cdot q + 2r + m \pmod{p})$

Intuition: Why Symmetric Decryption Works

Let's break down the decryption equation: $m' = (c \pmod{p}) \pmod{2}$.

1. Substitute c: $(p \cdot q + 2r + m \pmod{p})$
2. Because $p \cdot q$ is an exact multiple of p, it vanishes perfectly. We are left with $(2r + m \pmod{p})$.

Intuition: Why Symmetric Decryption Works

Let's break down the decryption equation: $m' = (c \pmod{p}) \pmod{2}$.

1. Substitute c: $(p \cdot q + 2r + m \pmod{p})$
2. Because $p \cdot q$ is an exact multiple of p, it vanishes perfectly. We are left with $(2r + m \pmod{p})$.
3. **The Critical Condition:** The modulo p operation only leaves $2r + m$ unchanged if its absolute value is strictly less than half the modulus. We strictly require:

$$|2r + m| < p/2$$

Intuition: Why Symmetric Decryption Works

Let's break down the decryption equation: $m' = (c \pmod{p}) \pmod{2}$.

1. Substitute c: $(p \cdot q + 2r + m \pmod{p})$
2. Because $p \cdot q$ is an exact multiple of p, it vanishes perfectly. We are left with $(2r + m \pmod{p})$.
3. **The Critical Condition:** The modulo p operation only leaves $2r + m$ unchanged if its absolute value is strictly less than half the modulus. We strictly require:

$$|2r + m| < p/2$$

4. Assuming the noise is small enough, we take $(2r + m) \pmod{2}$. The $2r$ is even, so it vanishes, leaving exactly m.

Intuition: Why Symmetric Decryption Works

Let's break down the decryption equation: $m' = (c \pmod{p}) \pmod{2}$.

1. Substitute c: $(p \cdot q + 2r + m \pmod{p})$
2. Because $p \cdot q$ is an exact multiple of p, it vanishes perfectly. We are left with $(2r + m \pmod{p})$.
3. **The Critical Condition:** The modulo p operation only leaves $2r + m$ unchanged if its absolute value is strictly less than half the modulus. We strictly require:

$$|2r + m| < p/2$$

4. Assuming the noise is small enough, we take $(2r + m) \pmod{2}$. The $2r$ is even, so it vanishes, leaving exactly m.

Motivation

To anyone without p, the ciphertext c just looks like a massive, random integer. But algebraically, it is a point "tethered" to a multiple of p by a short, noisy string $(2r + m)$.

Transitioning to Public-Key

A true FHE scheme requires public-key infrastructure. We achieve this by publishing a massive list of "encryptions of zero."

Transitioning to Public-Key

A true FHE scheme requires public-key infrastructure. We achieve this by publishing a massive list of "encryptions of zero."

New Parameters (based on security parameter λ):

- γ: Bit-length of the integers in the public key.
- τ: Number of integers in the public key.
- ρ': Secondary noise parameter for encryption $\rho' = \rho + \omega(\log \lambda)$.

Transitioning to Public-Key

A true FHE scheme requires public-key infrastructure. We achieve this by publishing a massive list of "encryptions of zero."

New Parameters (based on security parameter λ):

- γ: Bit-length of the integers in the public key.
- τ: Number of integers in the public key.
- ρ': Secondary noise parameter for encryption $\rho' = \rho + \omega(\log \lambda)$.

The Noise Distribution $\mathcal{D}_{\gamma,\rho}(p)$

To generate an "encryption of zero", choose $q \leftarrow \mathbb{Z} \cap [0, 2^\gamma/p)$ and $r \leftarrow \mathbb{Z} \cap (-2^\rho, 2^\rho)$. Output $x = pq + r$.

The Public-Key Algorithms

KeyGen:

- Sample $\tau + 1$ integers $x_0, x_1, \ldots, x_\tau$ from $\mathcal{D}_{\gamma,\rho}(p)$.
- Relabel so x_0 is the largest. Restart unless x_0 is odd and its noise $r_p(x_0)$ is even.
- Public key $pk = \langle x_0, x_1, \ldots, x_\tau \rangle$.

The Public-Key Algorithms

KeyGen:

- Sample $\tau + 1$ integers $x_0, x_1, \ldots, x_\tau$ from $\mathcal{D}_{\gamma,\rho}(p)$.
- Relabel so x_0 is the largest. Restart unless x_0 is odd and its noise $r_p(x_0)$ is even.
- Public key $pk = \langle x_0, x_1, \ldots, x_\tau \rangle$.

Encrypt(pk, m): Choose a random subset $S \subseteq \{1, 2, \ldots, \tau\}$ and a noise integer $r \in (-2^{\rho'}, 2^{\rho'})$.

$$c = \left[m + 2r + 2 \sum_{i \in S} x_i \right]_{x_0}$$

The Public-Key Algorithms

KeyGen:

- Sample $\tau + 1$ integers $x_0, x_1, \ldots, x_\tau$ from $\mathcal{D}_{\gamma,\rho}(p)$.
- Relabel so x_0 is the largest. Restart unless x_0 is odd and its noise $r_p(x_0)$ is even.
- Public key $pk = \langle x_0, x_1, \ldots, x_\tau \rangle$.

Encrypt(pk, m): Choose a random subset $S \subseteq \{1, 2, \ldots, \tau\}$ and a noise integer $r \in (-2^{\rho'}, 2^{\rho'})$.

$$c = \left[m + 2r + 2 \sum_{i \in S} x_i \right]_{x_0}$$

Intuition

Every x_i is effectively 0 (plus noise). Summing a random subset of x_i computes $0 + 0 + 0 \ldots$, creating a fresh, randomized encryption of zero. We multiply by 2 to keep the noise even, add our own fresh noise $2r$, and add m. The final modulo x_0 prevents the physical bit-size from growing infinitely large.

Lemma A.1: The Structure of a Fresh Ciphertext

To prove this scheme is homomorphic, we first mathematically prove that a fresh public-key ciphertext has the exact same structure as our symmetric toy scheme.

Lemma A.1: The Structure of a Fresh Ciphertext

To prove this scheme is homomorphic, we first mathematically prove that a fresh public-key ciphertext has the exact same structure as our symmetric toy scheme.

Lemma (A.1)

Let $c \leftarrow Encrypt(pk, m)$. *Then,* $c = a \cdot p + (2b + m)$ *for some integers* a *and* b *where the noise is bounded by* $|2b + m| \leq \tau 2^{\rho+2}$.

Lemma A.1: The Structure of a Fresh Ciphertext

To prove this scheme is homomorphic, we first mathematically prove that a fresh public-key ciphertext has the exact same structure as our symmetric toy scheme.

Lemma (A.1)

Let $c \leftarrow Encrypt(pk, m)$. Then, $c = a \cdot p + (2b + m)$ for some integers a and b where the noise is bounded by $|2b + m| \leq \tau 2^{\rho+2}$.

Proof Sketch:

- By definition, $c = m + 2r + 2\sum_{i \in S} x_i + k \cdot x_0$.
- We know every $x_i = q_i p + r_i$. Substitute this in:

$$c = p \cdot \left(kq_0 + 2\sum_{i \in S} q_i \right) + \left(m + 2r + 2k \cdot r_0 + 2\sum_{i \in S} r_i \right)$$

- The first term is a multiple of p (our $a \cdot p$). The second term is our accumulated noise $(2b + m)$. Its parity is m, and its magnitude is bounded.

Homomorphic Evaluation: Addition

Let the Evaluate algorithm simply be integer addition and multiplication. Let $c_1 = q_1 p + 2r_1 + m_1$ and $c_2 = q_2 p + 2r_2 + m_2$.

Homomorphic Evaluation: Addition

Let the Evaluate algorithm simply be integer addition and multiplication. Let $c_1 = q_1 p + 2r_1 + m_1$ and $c_2 = q_2 p + 2r_2 + m_2$.

Homomorphic Addition:

$$c_1 + c_2 = p(q_1 + q_2) + 2(r_1 + r_2) + (m_1 + m_2)$$

Homomorphic Evaluation: Addition

Let the Evaluate algorithm simply be integer addition and multiplication. Let $c_1 = q_1 p + 2r_1 + m_1$ and $c_2 = q_2 p + 2r_2 + m_2$.

Homomorphic Addition:

$$c_1 + c_2 = p(q_1 + q_2) + 2(r_1 + r_2) + (m_1 + m_2)$$

Observation: The structural format $a \cdot p +$ noise is perfectly preserved! We have a new multiple of p, plus $2 \times$ (new noise), plus the sum of the messages.

Crucially: The noise grows **linearly** $(r_1 + r_2)$.

Homomorphic Evaluation: Multiplication

Now, let us multiply the ciphertexts:

$$c_1 \cdot c_2 = (q_1 p + 2r_1 + m_1)(q_2 p + 2r_2 + m_2)$$

Homomorphic Evaluation: Multiplication

Now, let us multiply the ciphertexts:

$$c_1 \cdot c_2 = (q_1 p + 2r_1 + m_1)(q_2 p + 2r_2 + m_2)$$

Expanding this algebraically yields:

$$c_1 \cdot c_2 = p\big(p \cdot q_1 q_2 + q_1(2r_2 + m_2) + q_2(2r_1 + m_1)\big)$$

$$+ 2\big(2r_1 r_2 + r_1 m_2 + r_2 m_1\big) + m_1 m_2$$

Homomorphic Evaluation: Multiplication

Now, let us multiply the ciphertexts:

$$c_1 \cdot c_2 = (q_1 p + 2r_1 + m_1)(q_2 p + 2r_2 + m_2)$$

Expanding this algebraically yields:

$$c_1 \cdot c_2 = p\big(p \cdot q_1 q_2 + q_1(2r_2 + m_2) + q_2(2r_1 + m_1)\big)$$
$$+ 2\big(2r_1 r_2 + r_1 m_2 + r_2 m_1\big) + m_1 m_2$$

Observation: The structure is still preserved. We have a massive multiple of p (first term), plus $2 \times$ (mixed noise), plus the product of the messages $m_1 m_2$.

The Crisis: Look at the dominant noise term: $\mathbf{2r_1 r_2}$. When we multiply ciphertexts, the noise does not add—it **multiplies**.

The Decryption Constraint & The Noise Crisis

Lemma A.2 Constraint: Recall our strict rule for correct decryption. The modulo p operation ($c \pmod{p}$) will completely destroy the plaintext unless the absolute value of the total accumulated noise is strictly less than $p/2$.

The Decryption Constraint & The Noise Crisis

Lemma A.2 Constraint: Recall our strict rule for correct decryption. The modulo p operation ($c \pmod{p}$) will completely destroy the plaintext unless the absolute value of the total accumulated noise is strictly less than $p/2$.

The Crisis of the Square

Because multiplication squares the magnitude of the noise, the noise bit-length doubles with every sequential multiplication. It grows exponentially with the multiplicative depth of the circuit. We will rapidly hit the $p/2$ boundary.

Formalizing the Bottleneck: Lemma 3.5

Exactly how deep can our circuit be before the noise destroys the data?

Formalizing the Bottleneck: Lemma 3.5

Exactly how deep can our circuit be before the noise destroys the data?

Lemma (3.5: Permitted Polynomials)

Let $f(x_1, \ldots, x_t)$ be the multivariate polynomial computed by a circuit, and let d be its degree. The scheme can correctly evaluate this polynomial as long as:

$$d \leq \frac{\eta - 4 - \log ||\vec{f}||_1}{\rho' + 2}$$

where $||\vec{f}||_1$ is the l_1 norm of the coefficient vector of f.

Formalizing the Bottleneck: Lemma 3.5

Exactly how deep can our circuit be before the noise destroys the data?

Lemma (3.5: Permitted Polynomials)

Let $f(x_1, \ldots, x_t)$ be the multivariate polynomial computed by a circuit, and let d be its degree. The scheme can correctly evaluate this polynomial as long as:

$$d \leq \frac{\eta - 4 - \log ||\vec{f}||_1}{\rho' + 2}$$

where $||\vec{f}||_1$ is the l_1 norm of the coefficient vector of f.

Intuition: The Noise Budget

- Think of η (the bit-size of p) as our total **Noise Budget**.
- Think of ρ' (the initial noise size) as the **cost** of one multiplication.
- Therefore, the maximum multiplicative depth (degree d) we can handle is roughly our total budget divided by the cost per operation: $\approx \eta/\rho'$.

Conclusion of Phase 2

Where do we stand?

Conclusion of Phase 2

Where do we stand?

1. We have successfully built an encryption scheme that can evaluate additions and multiplications without the server knowing the underlying data.

Conclusion of Phase 2

Where do we stand?

1. We have successfully built an encryption scheme that can evaluate additions and multiplications without the server knowing the underlying data.
2. However, because of the multiplicative noise bottleneck, this scheme is strictly a **Somewhat Homomorphic Encryption (SHE)** scheme.

Conclusion of Phase 2

Where do we stand?

1. We have successfully built an encryption scheme that can evaluate additions and multiplications without the server knowing the underlying data.
2. However, because of the multiplicative noise bottleneck, this scheme is strictly a **Somewhat Homomorphic Encryption (SHE)** scheme.
3. If we evaluate a circuit deeper than $d \approx \eta/\rho'$, the noise wraps around the modulus p, and the data is lost forever.

Conclusion of Phase 2

Where do we stand?

1. We have successfully built an encryption scheme that can evaluate additions and multiplications without the server knowing the underlying data.
2. However, because of the multiplicative noise bottleneck, this scheme is strictly a **Somewhat Homomorphic Encryption (SHE)** scheme.
3. If we evaluate a circuit deeper than $d \approx \eta/\rho'$, the noise wraps around the modulus p, and the data is lost forever.

To achieve our dream of infinite FHE computation, we must find a way to evaluate our own decryption circuit within this strict noise budget. This leads us to Phase 3...

References

Marten van Dijk, Craig Gentry, Shai Halevi, and Vinod Vaikuntanathan. *Fully Homomorphic Encryption over the Integers*. Eurocrypt, 2010.

Phase 3: Variations and Optimizations

Fully Homomorphic Encryption over the Integers

Paper Presentation

April 13, 2026

Outline: Phase 3

1. The Practicality Gap
2. Optimization 1: Modular Reduction During Evaluate
3. Optimization 2: Ciphertext Compression
4. References

The Practicality Gap

Motivation

We have established the mathematical engine for our Somewhat Homomorphic Encryption (SHE) scheme. However, before we can secure it or bootstrap it, we face two massive engineering realities:

1. **Memory Explosion:** *Ciphertexts grow exponentially in physical bit-size during evaluation.*
2. **Bandwidth Choke:** *Even fresh ciphertexts are impractically large ($\tilde{\mathcal{O}}(\lambda^5)$ bits).*

The Practicality Gap

Motivation

We have established the mathematical engine for our Somewhat Homomorphic Encryption (SHE) scheme. However, before we can secure it or bootstrap it, we face two massive engineering realities:

1. **Memory Explosion:** *Ciphertexts grow exponentially in physical bit-size during evaluation.*
2. **Bandwidth Choke:** *Even fresh ciphertexts are impractically large ($\tilde{\mathcal{O}}(\lambda^5)$ bits).*

To make this scheme practically viable, we must introduce two structural optimizations:

- *In-flight:* Modular reduction during evaluation.
- *Post-flight:* Ciphertext compression via group exponentiation.

The Size Explosion Problem

Recall our encryption algorithm: $c = \left[m + 2r + 2\sum_{i \in S} x_i\right]_{x_0}$.

The Size Explosion Problem

Recall our encryption algorithm: $c = \left[m + 2r + 2\sum_{i \in S} x_i\right]_{x_0}$.

The Problem: During Encrypt, we easily keep the ciphertext size bounded to γ bits by reducing modulo x_0.
However, during Evaluate, we **cannot** simply reduce modulo x_0 after a multiplication.

The Size Explosion Problem

Recall our encryption algorithm: $c = \left[m + 2r + 2\sum_{i \in S} x_i\right]_{x_0}$.

The Problem: During Encrypt, we easily keep the ciphertext size bounded to γ bits by reducing modulo x_0.
However, during Evaluate, we **cannot** simply reduce modulo x_0 after a multiplication.

Why? After one multiplication, the ciphertext squares in size, becoming vastly larger than x_0. A standard modulo x_0 reduction would subtract a massive multiple of x_0. Because x_0 contains its own noise (r_0), subtracting a massive multiple of x_0 injects a catastrophic amount of noise into the ciphertext, instantly destroying the plaintext.

The Solution: A Ladder of Zeroes

The Fix: We augment the public key with a "ladder" of increasingly massive encryptions of zero.

The Solution: A Ladder of Zeroes

The Fix: We augment the public key with a "ladder" of increasingly massive encryptions of zero.

For $i = 0, \ldots, \gamma$, we generate new elements x'_i:

$$q'_i \leftarrow \mathbb{Z} \cap [2^{\gamma+i-1}/p, 2^{\gamma+i}/p)$$

$$x'_i = 2(q'_i \cdot p + r'_i)$$

where r'_i is drawn from the standard noise distribution $(-2^\rho, 2^\rho)$.

The Solution: A Ladder of Zeroes

The Fix: We augment the public key with a "ladder" of increasingly massive encryptions of zero.

For $i = 0, \ldots, \gamma$, we generate new elements x'_i:

$$q'_i \leftarrow \mathbb{Z} \cap [2^{\gamma+i-1}/p, 2^{\gamma+i}/p)$$

$$x'_i = 2(q'_i \cdot p + r'_i)$$

where r'_i is drawn from the standard noise distribution $(-2^\rho, 2^\rho)$.

Intuition (The Step-Down Ladder)

These x'_i values are astronomically large, scaling up to $2^{2\gamma}$.
During evaluation, when a ciphertext exceeds 2^γ, we do not reduce by x_0. Instead, we reduce modulo the largest x'_γ, then modulo $x'_{\gamma-1}$, and so on, stepping down the ladder until the ciphertext is back to γ bits.

The Solution: A Ladder of Zeroes

The Fix: We augment the public key with a "ladder" of increasingly massive encryptions of zero.

For $i = 0, \ldots, \gamma$, we generate new elements x'_i:

$$q'_i \leftarrow \mathbb{Z} \cap [2^{\gamma+i-1}/p, 2^{\gamma+i}/p)$$

$$x'_i = 2(q'_i \cdot p + r'_i)$$

where r'_i is drawn from the standard noise distribution $(-2^\rho, 2^\rho)$.

Intuition (The Step-Down Ladder)

These x'_i values are astronomically large, scaling up to $2^{2\gamma}$.
During evaluation, when a ciphertext exceeds 2^γ, we do not reduce by x_0. Instead, we reduce modulo the largest x'_γ, then modulo $x'_{\gamma-1}$, and so on, stepping down the ladder until the ciphertext is back to γ bits.

Result: Because we only ever subtract small multiples of these x'_i elements at each step, the accumulated error remains strictly bounded!

The Bandwidth Problem

Even with the ladder optimization keeping ciphertexts at γ bits, γ is still massively large—specifically $\tilde{\mathcal{O}}(\lambda^5)$ bits. Transmitting this over a network is brutally inefficient.

The Bandwidth Problem

Even with the ladder optimization keeping ciphertexts at γ bits, γ is still massively large—specifically $\tilde{\mathcal{O}}(\lambda^5)$ bits. Transmitting this over a network is brutally inefficient.

The Goal: Compress the final output ciphertext to the size of a standard RSA modulus *after* the cloud server has finished all homomorphic evaluations.

Compression via Group Exponentiation

The Mechanism: We supplement the public key with the description of a cyclic group $G = \langle g \rangle$ whose order is a multiple of the secret key p.

Compression via Group Exponentiation

The Mechanism: We supplement the public key with the description of a cyclic group $G = \langle g \rangle$ whose order is a multiple of the secret key p.

The Compression Step: Instead of sending the massive integer c back to the user, the server computes and transmits:

$$c^* = g^c \pmod{N}$$

Compression via Group Exponentiation

The Mechanism: We supplement the public key with the description of a cyclic group $G = \langle g \rangle$ whose order is a multiple of the secret key p.

The Compression Step: Instead of sending the massive integer c back to the user, the server computes and transmits:

$$c^* = g^c \pmod{N}$$

The Decryption Step: Because c^* is an element of the group, the user decrypts by computing the Discrete Logarithm (DL):

$$m = (DL_g(c^*) \pmod{p}) \pmod{2}$$

Trade-offs of Compression

How is computing the Discrete Logarithm efficient for the user?

Trade-offs of Compression

How is computing the Discrete Logarithm efficient for the user?

The Secret Key Modification: We must generate the secret key p as a **smooth** number (a product of many small distinct primes). This allows the user to efficiently compute the discrete log using the Pohlig-Hellman algorithm or baby-step giant-step method.

Trade-offs of Compression

How is computing the Discrete Logarithm efficient for the user?

The Secret Key Modification: We must generate the secret key p as a **smooth** number (a product of many small distinct primes). This allows the user to efficiently compute the discrete log using the Pohlig-Hellman algorithm or baby-step giant-step method.

The Ultimate Trade-off

- **Pros:** The ciphertext is compressed from $\tilde{\mathcal{O}}(\lambda^5)$ bits down to the size of an RSA modulus (~ 2048 bits).
- **Cons:** The compressed ciphertext c^* resides in the group G. Because we do not have an FHE scheme over this group, **we completely lose all homomorphic properties.**

Trade-offs of Compression

How is computing the Discrete Logarithm efficient for the user?

The Secret Key Modification: We must generate the secret key p as a **smooth** number (a product of many small distinct primes). This allows the user to efficiently compute the discrete log using the Pohlig-Hellman algorithm or baby-step giant-step method.

The Ultimate Trade-off

- **Pros:** The ciphertext is compressed from $\tilde{\mathcal{O}}(\lambda^5)$ bits down to the size of an RSA modulus (~ 2048 bits).
- **Cons:** The compressed ciphertext c^* resides in the group G. Because we do not have an FHE scheme over this group, **we completely lose all homomorphic properties.**

Conclusion: This is strictly a post-flight optimization, applied only when computation is complete and data is ready for transit.

References

- Marten van Dijk, Craig Gentry, Shai Halevi, and Vinod Vaikuntanathan. *Fully Homomorphic Encryption over the Integers*. Eurocrypt, 2010.
- C. Cachin, S. Micali, and M. Stadler. *Computationally private information retrieval with polylogarithmic communication*. EUROCRYPT, 1999. (For the ϕ-hiding assumption).

Phase 4: Security Reduction and Known Attacks

Fully Homomorphic Encryption over the Integers

Paper Presentation

April 14, 2026

Outline: Phase 4

1. The Approximate-GCD Problem
2. The Security Reduction (Outline)
3. Why Standard Attacks Fail
4. The Geometric Threat: Lattice Attacks
5. References

The Need for Provable Security

Motivation

We have established a working mathematical engine, but in modern cryptography, we do not simply guess that a scheme is secure. We must rigorously prove that breaking the semantic security of our encryption is mathematically equivalent to solving a known, well-studied hard problem.

The Need for Provable Security

Motivation

We have established a working mathematical engine, but in modern cryptography, we do not simply guess that a scheme is secure. We must rigorously prove that breaking the semantic security of our encryption is mathematically equivalent to solving a known, well-studied hard problem.

For RSA, that problem is integer factorization. For Gentry's 2009 FHE, it is finding short vectors in ideal lattices.

The Need for Provable Security

Motivation

We have established a working mathematical engine, but in modern cryptography, we do not simply guess that a scheme is secure. We must rigorously prove that breaking the semantic security of our encryption is mathematically equivalent to solving a known, well-studied hard problem.

For RSA, that problem is integer factorization. For Gentry's 2009 FHE, it is finding short vectors in ideal lattices.

For our integer-based scheme, the foundation is the **Approximate-GCD Problem**.

Defining Approximate-GCD

Recall our noise distribution $\mathcal{D}_{\gamma,\rho}(p)$, which outputs integers of the form $x = pq + r$, where x is roughly γ bits long, and the noise r is bounded by 2^{ρ}.

Defining Approximate-GCD

Recall our noise distribution $\mathcal{D}_{\gamma,\rho}(p)$, which outputs integers of the form $x = pq + r$, where x is roughly γ bits long, and the noise r is bounded by 2^ρ.

Definition (Approximate Integer Greatest Common Divisor)

The (ρ, η, γ)-approximate-gcd problem is: given polynomially many samples $x_0, x_1, \ldots, x_\tau$ drawn from $\mathcal{D}_{\gamma,\rho}(p)$ for a randomly chosen η-bit odd integer p, output the hidden integer p.

Defining Approximate-GCD

Recall our noise distribution $\mathcal{D}_{\gamma,\rho}(p)$, which outputs integers of the form $x = pq + r$, where x is roughly γ bits long, and the noise r is bounded by 2^ρ.

Definition (Approximate Integer Greatest Common Divisor)

The (ρ, η, γ)-approximate-gcd problem is: given polynomially many samples $x_0, x_1, \ldots, x_\tau$ drawn from $\mathcal{D}_{\gamma,\rho}(p)$ for a randomly chosen η-bit odd integer p, output the hidden integer p.

Intuition

If the noise $r = 0$, the problem is trivial. You are given exact multiples of p, and a single run of the 2,000-year-old Euclidean algorithm computes $\gcd(x_1, x_2) = p$ instantly. The addition of the noise term r is the sole cryptographic barrier.

Theorem 4.2: The Search-to-Decision Reduction

Motivation

We must mathematically prove that breaking our scheme is equivalent to solving the Approximate-GCD problem. We do this via a "Search-to-Decision" reduction.

Theorem 4.2: The Search-to-Decision Reduction

Motivation

We must mathematically prove that breaking our scheme is equivalent to solving the Approximate-GCD problem. We do this via a "Search-to-Decision" reduction.

Theorem (4.2)

Any attack $\mathcal{A}$ with advantage ϵ on the semantic security of the encryption scheme can be converted into an algorithm $\mathcal{B}$ for solving the (ρ, η, γ)-approximate-gcd problem with success probability at least $\epsilon/2$.

Theorem 4.2: The Search-to-Decision Reduction

Motivation

We must mathematically prove that breaking our scheme is equivalent to solving the Approximate-GCD problem. We do this via a "Search-to-Decision" reduction.

Theorem (4.2)

Any attack $\mathcal{A}$ with advantage ϵ on the semantic security of the encryption scheme can be converted into an algorithm $\mathcal{B}$ for solving the (ρ, η, γ)-approximate-gcd problem with success probability at least $\epsilon/2$.

The Setup: Algorithm $\mathcal{B}$ is given random noisy multiples of a hidden integer p. Its goal is to find p. To do this, $\mathcal{B}$ uses the attacker $\mathcal{A}$ as a tool, feeding $\mathcal{A}$ simulated public keys and ciphertexts to extract information about p.

The Mechanism: Oracle and Binary GCD

How does $\mathcal{B}$ use the attacker $\mathcal{A}$ to find the secret key p?

The Mechanism: Oracle and Binary GCD

How does $\mathcal{B}$ use the attacker $\mathcal{A}$ to find the secret key p?

Step 1: The Learn-LSB Oracle

- $\mathcal{B}$ constructs randomized, simulated ciphertexts from a noisy integer z.
- By observing whether $\mathcal{A}$ correctly guesses the encrypted bits, $\mathcal{B}$ can deduce the **parity of the quotient** $q_p(z)$, without ever knowing p!

The Mechanism: Oracle and Binary GCD

How does $\mathcal{B}$ use the attacker $\mathcal{A}$ to find the secret key p?

Step 1: The Learn-LSB Oracle

- $\mathcal{B}$ constructs randomized, simulated ciphertexts from a noisy integer z.
- By observing whether $\mathcal{A}$ correctly guesses the encrypted bits, $\mathcal{B}$ can deduce the **parity of the quotient** $q_p(z)$, without ever knowing p!

Step 2: The Binary GCD Extraction

- $\mathcal{B}$ takes two noisy multiples of p and runs a standard Binary-GCD algorithm on them.
- Whenever the algorithm needs to know if a value is even or odd, $\mathcal{B}$ queries the Learn-LSB Oracle.
- **Result:** Iterating this process strips away the noise terms completely, reducing the values down to the exact secret key p.

Guaranteeing Success: The Distribution Match

For this reduction to hold, we must guarantee that attacker $\mathcal{A}$ cannot tell that it is being manipulated by $\mathcal{B}$. The simulated ciphertexts must look perfectly real.

Guaranteeing Success: The Distribution Match

For this reduction to hold, we must guarantee that attacker $\mathcal{A}$ cannot tell that it is being manipulated by $\mathcal{B}$. The simulated ciphertexts must look perfectly real.

Why $\mathcal{A}$ is successfully fooled (Section 4.1.1):

1. **Quotient Uniformity:** By applying the **Leftover Hash Lemma**, the sum of the random subsets used to build the simulated ciphertext makes the underlying quotient statistically uniform.

Guaranteeing Success: The Distribution Match

For this reduction to hold, we must guarantee that attacker $\mathcal{A}$ cannot tell that it is being manipulated by $\mathcal{B}$. The simulated ciphertexts must look perfectly real.

Why $\mathcal{A}$ is successfully fooled (Section 4.1.1):

1. **Quotient Uniformity:** By applying the **Leftover Hash Lemma**, the sum of the random subsets used to build the simulated ciphertext makes the underlying quotient statistically uniform.
2. **Noise Drowning:** $\mathcal{B}$ injects a massive random noise term (drawn from $2^{\rho'}$) into the simulation. This completely "drowns out" and statistically hides any underlying discrepancies in the base noise.

Guaranteeing Success: The Distribution Match

For this reduction to hold, we must guarantee that attacker $\mathcal{A}$ cannot tell that it is being manipulated by $\mathcal{B}$. The simulated ciphertexts must look perfectly real.

Why $\mathcal{A}$ is successfully fooled (Section 4.1.1):

1. **Quotient Uniformity:** By applying the **Leftover Hash Lemma**, the sum of the random subsets used to build the simulated ciphertext makes the underlying quotient statistically uniform.
2. **Noise Drowning:** $\mathcal{B}$ injects a massive random noise term (drawn from $2^{\rho'}$) into the simulation. This completely "drowns out" and statistically hides any underlying discrepancies in the base noise.

Intuition

Because the simulated data is statistically indistinguishable from real FHE traffic, $\mathcal{A}$'s advantage holds. A rigorous counting argument proves that $\mathcal{B}$ will successfully recover the Approximate-GCD secret key p with probability $\geq \epsilon/2$.

Algebraic Failures: Continued Fractions

If we cannot break the semantic security, can we attack the Approximate-GCD directly?

Attempt 1: Continued Fractions Since $x_1 \approx q_1 p$ and $x_2 \approx q_2 p$, we know the public ratio x_1/x_2 is very close to the unknown rational q_1/q_2. Can we find q_1/q_2 in the continued fraction expansion of x_1/x_2?

Algebraic Failures: Continued Fractions

If we cannot break the semantic security, can we attack the Approximate-GCD directly?

Attempt 1: Continued Fractions Since $x_1 \approx q_1 p$ and $x_2 \approx q_2 p$, we know the public ratio x_1/x_2 is very close to the unknown rational q_1/q_2. Can we find q_1/q_2 in the continued fraction expansion of x_1/x_2?

The Mathematical Failure: Let's analyze the distance between the public ratio and the target quotient ratio:

$$\left|\frac{x_1}{x_2} - \frac{q_1}{q_2}\right| = \left|\frac{q_2(q_1 p + r_1) - q_1(q_2 p + r_2)}{q_2(q_2 p + r_2)}\right| \approx \left|\frac{q_2 r_1 - q_1 r_2}{p \cdot q_2^2}\right|$$

Algebraic Failures: Continued Fractions

If we cannot break the semantic security, can we attack the Approximate-GCD directly?

Attempt 1: Continued Fractions Since $x_1 \approx q_1 p$ and $x_2 \approx q_2 p$, we know the public ratio x_1/x_2 is very close to the unknown rational q_1/q_2. Can we find q_1/q_2 in the continued fraction expansion of x_1/x_2?

The Mathematical Failure: Let's analyze the distance between the public ratio and the target quotient ratio:

$$\left|\frac{x_1}{x_2} - \frac{q_1}{q_2}\right| = \left|\frac{q_2(q_1 p + r_1) - q_1(q_2 p + r_2)}{q_2(q_2 p + r_2)}\right| \approx \left|\frac{q_2 r_1 - q_1 r_2}{p \cdot q_2^2}\right|$$

The Cross-Noise Barrier

The distance is dominated by the cross-multiplied noise term $(q_2 r_1 - q_1 r_2)/p$. Because this distance is significantly larger than $1/q_2^2$, Dirichlet's approximation theorem does not apply. The target fraction is hopelessly obscured by the noise.

The Simultaneous Diophantine Approximation (SDA)

Because simple algebraic attacks fail, attackers must turn to the geometry of numbers—specifically lattice reduction algorithms like LLL and BKZ.

The Simultaneous Diophantine Approximation (SDA)

Because simple algebraic attacks fail, attackers must turn to the geometry of numbers—specifically lattice reduction algorithms like LLL and BKZ.

Given samples $x_0, x_1, \ldots, x_t$, we construct a $(t+1)$-dimensional SDA lattice L spanned by the rows of matrix M:

$$M = \begin{pmatrix} 2^\rho & 0 & 0 & \ldots & 0 \\ x_1 & -x_0 & 0 & \ldots & 0 \\ x_2 & 0 & -x_0 & \ldots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ x_t & 0 & 0 & \ldots & -x_0 \end{pmatrix}$$

Finding the Target Vector

The attacker's goal is to find the hidden vector of quotients $\vec{q} = \langle q_0, q_1, \ldots, q_t \rangle$. Let's multiply $\vec{q}$ by M:

$$\vec{v} = \vec{q} \cdot M = \langle q_0 2^\rho,\ q_0 x_1 - q_1 x_0,\ \ldots,\ q_0 x_t - q_t x_0 \rangle$$

Finding the Target Vector

The attacker's goal is to find the hidden vector of quotients $\vec{q} = \langle q_0, q_1, \ldots, q_t \rangle$. Let's multiply $\vec{q}$ by M:

$$\vec{v} = \vec{q} \cdot M = \langle q_0 2^\rho,\ q_0 x_1 - q_1 x_0,\ \ldots,\ q_0 x_t - q_t x_0 \rangle$$

The Critical Cancellation: Look at the components $q_0 x_i - q_i x_0$. Since $x_i = q_i p + r_i$, we can expand:

$$q_0 x_i - q_i x_0 = q_0(q_i p + r_i) - q_i(q_0 p + r_0)$$

$$= q_0 q_i p + q_0 r_i - q_i q_0 p - q_i r_0$$

$$= \mathbf{q_0 r_i - q_i r_0}$$

Finding the Target Vector

The attacker's goal is to find the hidden vector of quotients $\vec{q} = \langle q_0, q_1, \ldots, q_t \rangle$. Let's multiply $\vec{q}$ by M:

$$\vec{v} = \vec{q} \cdot M = \langle q_0 2^\rho,\ q_0 x_1 - q_1 x_0,\ \ldots,\ q_0 x_t - q_t x_0 \rangle$$

The Critical Cancellation: Look at the components $q_0 x_i - q_i x_0$. Since $x_i = q_i p + r_i$, we can expand:

$$q_0 x_i - q_i x_0 = q_0(q_i p + r_i) - q_i(q_0 p + r_0)$$

$$= q_0 q_i p + q_0 r_i - q_i q_0 p - q_i r_0$$

$$= \mathbf{q_0 r_i - q_i r_0}$$

The Threat

The secret key p perfectly cancels out! This leaves $\vec{v}$ as a relatively short target vector in the lattice. If an algorithm like LLL can find $\vec{v}$, the attacker recovers the quotients, computes $p \approx x_0/q_0$, and the scheme is broken.

Defeating the Lattice

Will LLL or BKZ actually find this target vector $\vec{v}$? This is where our parameter selection defends the scheme.

Defeating the Lattice

Will LLL or BKZ actually find this target vector $\vec{v}$? This is where our parameter selection defends the scheme.

Scenario 1: Attacker uses a small number of samples t

- By Minkowski's bound, a low-dimensional lattice contains exponentially many *spurious* vectors that are much shorter than our target $\vec{v}$.
- The target vector is completely buried in geometric noise.

Defeating the Lattice

Will LLL or BKZ actually find this target vector $\vec{v}$? This is where our parameter selection defends the scheme.

Scenario 1: Attacker uses a small number of samples t

- By Minkowski's bound, a low-dimensional lattice contains exponentially many *spurious* vectors that are much shorter than our target $\vec{v}$.
- The target vector is completely buried in geometric noise.

Scenario 2: Attacker uses a massive number of samples t

- The attacker forces t to be huge so $\vec{v}$ is the absolute shortest vector.
- However, known lattice reduction algorithms take time $\approx 2^{t/k}$ to find a 2^k approximation.
- Isolating $\vec{v}$ strictly requires time roughly $\mathbf{2}^{\gamma/\eta^2}$.

Summary of Secure Parameters

By setting the bit-length of the public key elements (γ) to be massively large, we force the lattice dimension to be computationally impossible to reduce.

Summary of Secure Parameters

By setting the bit-length of the public key elements (γ) to be massively large, we force the lattice dimension to be computationally impossible to reduce.

Final Secure Parameters:

- $\rho = \mathcal{O}(\lambda)$: Prevents brute-force guessing of the noise r.
- $\eta = \tilde{\mathcal{O}}(\lambda^2)$: Supports the homomorphic noise budget (depth for bootstrapping).
- $\gamma = \tilde{\mathcal{O}}(\lambda^5)$: Specifically ensures that $\gamma = \omega(\eta^2 \log \lambda)$. This makes the time $2^{\gamma/\eta^2}$ super-polynomial, thwarting all SDA and lattice reduction algorithms.

Summary of Secure Parameters

By setting the bit-length of the public key elements (γ) to be massively large, we force the lattice dimension to be computationally impossible to reduce.

Final Secure Parameters:

- $\rho = \mathcal{O}(\lambda)$: Prevents brute-force guessing of the noise r.
- $\eta = \tilde{\mathcal{O}}(\lambda^2)$: Supports the homomorphic noise budget (depth for bootstrapping).
- $\gamma = \tilde{\mathcal{O}}(\lambda^5)$: Specifically ensures that $\gamma = \omega(\eta^2 \log \lambda)$. This makes the time $2^{\gamma/\eta^2}$ super-polynomial, thwarting all SDA and lattice reduction algorithms.

Conclusion: While this results in massive public keys, it mathematically guarantees rigorous security based solely on the hardness of the Approximate-GCD over the integers.

References

- Marten van Dijk, Craig Gentry, Shai Halevi, and Vinod Vaikuntanathan. *Fully Homomorphic Encryption over the Integers*. Eurocrypt, 2010.
- J. C. Lagarias. *The computational complexity of simultaneous diophantine approximation problems*. SIAM J. Comput., 1985.
- N. Howgrave-Graham. *Approximate integer common divisors*. CaLC, 2001.

Phase 5: The Climax – Squashing and Achieving FHE

Fully Homomorphic Encryption over the Integers

Paper Presentation

April 14, 2026

Outline: Phase 5

1. The Depth Problem
2. Squashing the Decryption Circuit
3. The Shallow Circuit & Bootstrapping Achieved
4. References

The Crisis of Bootstrapping

Motivation

We established in Phase 1 that to achieve Fully Homomorphic Encryption, our scheme must be **bootstrappable**. *It must be able to homomorphically evaluate its own decryption circuit before the noise exceeds the $p/2$ boundary.*

The Crisis of Bootstrapping

Motivation

We established in Phase 1 that to achieve Fully Homomorphic Encryption, our scheme must be **bootstrappable**. *It must be able to homomorphically evaluate its own decryption circuit before the noise exceeds the $p/2$ boundary.*

Let us look at the mathematics of our decryption equation:

$$m' = (c \pmod{p}) \pmod{2}$$

The Crisis of Bootstrapping

Motivation

We established in Phase 1 that to achieve Fully Homomorphic Encryption, our scheme must be **bootstrappable**. *It must be able to homomorphically evaluate its own decryption circuit before the noise exceeds the $p/2$ boundary.*

Let us look at the mathematics of our decryption equation:

$$m' = (c \pmod{p}) \pmod{2}$$

To evaluate this homomorphically as a boolean circuit, we must compute it over the integers:

$$c \pmod{p} = c - p\lfloor c/p \rceil$$

The Wall of Integer Division

The Roadblock: To evaluate $c - p\lfloor c/p \rceil$, the circuit must compute $\lfloor c/p \rceil$. This requires **integer division**.

The Wall of Integer Division

The Roadblock: To evaluate $c - p\lfloor c/p \rceil$, the circuit must compute $\lfloor c/p \rceil$. This requires **integer division**.

When translated into a boolean polynomial circuit, division has a massive multiplicative depth.

The Wall of Integer Division

The Roadblock: To evaluate $c - p\lfloor c/p \rceil$, the circuit must compute $\lfloor c/p \rceil$. This requires **integer division**.

When translated into a boolean polynomial circuit, division has a massive multiplicative depth.

Recall our permitted polynomial degree bound d from Phase 2:

$$d \leq \frac{\eta - 4 - \log ||\hat{f}||_1}{\rho' + 2}$$

The Wall of Integer Division

The Roadblock: To evaluate $c - p\lfloor c/p \rceil$, the circuit must compute $\lfloor c/p \rceil$. This requires **integer division**.

When translated into a boolean polynomial circuit, division has a massive multiplicative depth.

Recall our permitted polynomial degree bound d from Phase 2:

$$d \leq \frac{\eta - 4 - \log ||\hat{f}||_1}{\rho' + 2}$$

The Reality: The standard division circuit's degree far exceeds this permitted noise budget. The decryption circuit is simply too deep. We must **"squash"** it.

Squashing: Shifting the Burden

To make the scheme bootstrappable, we must reduce the multiplicative depth of the decryption circuit.

Squashing: Shifting the Burden

To make the scheme bootstrappable, we must reduce the multiplicative depth of the decryption circuit.

Intuition (The Squashing Strategy)

Division by p is mathematically expensive. Multiplication is relatively cheap. We will shift the heavy computational lifting from the Decrypt algorithm (which is strictly constrained by the noise budget) to the Evaluate algorithm (which can be done in the clear by the server before the final homomorphic step).

Squashing: Shifting the Burden

To make the scheme bootstrappable, we must reduce the multiplicative depth of the decryption circuit.

Intuition (The Squashing Strategy)

Division by p is mathematically expensive. Multiplication is relatively cheap. We will shift the heavy computational lifting from the Decrypt algorithm (which is strictly constrained by the noise budget) to the Evaluate algorithm (which can be done in the clear by the server before the final homomorphic step).

New Parameters:

- κ: Precision of rational numbers.
- Θ: Size of a massive public set of rational numbers.
- θ: The Hamming weight of a sparse secret subset.

Modifying Key Generation: The Hint

During KeyGen, alongside p and pk^*, we generate a "hint" about the value of $1/p$.

Modifying Key Generation: The Hint

During KeyGen, alongside p and pk^*, we generate a "hint" about the value of $1/p$.

1. The New Secret Key ($\vec{s}$): Choose a random Θ-bit vector $\vec{s} = \langle s_1, \ldots, s_\Theta \rangle$ with a very low Hamming weight θ. Let $S = \{i : s_i = 1\}$. This sparse vector $\vec{s}$ becomes the new secret key.

Modifying Key Generation: The Hint

During KeyGen, alongside p and pk^*, we generate a "hint" about the value of $1/p$.

1. The New Secret Key ($\vec{s}$): Choose a random Θ-bit vector $\vec{s} = \langle s_1, \ldots, s_\Theta \rangle$ with a very low Hamming weight θ. Let $S = \{i : s_i = 1\}$. This sparse vector $\vec{s}$ becomes the new secret key.

2. The Public Rational Numbers ($\vec{y}$): Generate a set $\vec{y} = \{y_1, \ldots, y_\Theta\}$ of rational numbers in $[0, 2)$ with κ bits of precision, such that the subset S sums to approximately $1/p$:

$$\sum_{i \in S} y_i \approx \frac{1}{p} \pmod 2$$

Specifically, $\left[\sum_{i \in S} y_i\right]_2 = \frac{1}{p} - \Delta_p$, where $|\Delta_p| < 2^{-\kappa}$.

Modifying Evaluate and Decrypt

When the server finishes homomorphically evaluating a circuit, it holds a noisy ciphertext c^*. **Before returning it**, the server does post-processing in the clear.

Modifying Evaluate and Decrypt

When the server finishes homomorphically evaluating a circuit, it holds a noisy ciphertext c^*. **Before returning it**, the server does post-processing in the clear.

Server Post-Processing: For $i \in \{1, \ldots, \Theta\}$, the server computes:

$$z_i = [c^* \cdot y_i]_2$$

keeping only $n = \lceil \log \theta \rceil + 3$ bits of precision. The server outputs c^* and $\vec{z} = \langle z_1, \ldots, z_\Theta \rangle$.

Modifying Evaluate and Decrypt

When the server finishes homomorphically evaluating a circuit, it holds a noisy ciphertext c^*. **Before returning it**, the server does post-processing in the clear.

Server Post-Processing: For $i \in \{1, \ldots, \Theta\}$, the server computes:

$$z_i = [c^* \cdot y_i]_2$$

keeping only $n = \lceil \log \theta \rceil + 3$ bits of precision. The server outputs c^* and $\vec{z} = \langle z_1, \ldots, z_\Theta \rangle$.

Deriving the Squashed Decryption: We need to compute $c^*/p \pmod 2$.

$$\frac{c^*}{p} \approx c^* \sum_{i \in S} y_i \pmod 2 = \sum_{i=1}^{\Theta} s_i (c^* y_i) \pmod 2 \approx \sum_{i=1}^{\Theta} s_i z_i \pmod 2$$

Modifying Evaluate and Decrypt

When the server finishes homomorphically evaluating a circuit, it holds a noisy ciphertext c^*. **Before returning it**, the server does post-processing in the clear.

Server Post-Processing: For $i \in \{1, \ldots, \Theta\}$, the server computes:

$$z_i = [c^* \cdot y_i]_2$$

keeping only $n = \lceil \log \theta \rceil + 3$ bits of precision. The server outputs c^* and $\vec{z} = \langle z_1, \ldots, z_\Theta \rangle$.

Deriving the Squashed Decryption: We need to compute $c^*/p \pmod 2$.

$$\frac{c^*}{p} \approx c^* \sum_{i \in S} y_i \pmod 2 = \sum_{i=1}^{\Theta} s_i (c^* y_i) \pmod 2 \approx \sum_{i=1}^{\Theta} s_i z_i \pmod 2$$

The New Decrypt Algorithm:

$$m' = \left[c^* - \left\lfloor \sum_{i=1}^{\Theta} s_i z_i \right\rceil \right]_2$$

The Shallow Decryption Circuit

Look at our new decryption equation: $m' = \left[c^* - \lfloor \sum_{i=1}^{\Theta} s_i z_i \rceil\right]_2$.

The Shallow Decryption Circuit

Look at our new decryption equation: $m' = \left[c^* - \lfloor \sum_{i=1}^{\Theta} s_i z_i \rceil \right]_2$.

Why is this a mathematical triumph?

- The server has already provided the rational numbers z_i.
- The secret key bits s_i are strictly 0 or 1.
- Our only job is to sum up numbers. Because $\vec{s}$ is sparse, only θ of them are non-zero!

The Shallow Decryption Circuit

Look at our new decryption equation: $m' = \left[c^* - \lfloor \sum_{i=1}^{\Theta} s_i z_i \rceil \right]_2$.

Why is this a mathematical triumph?

- The server has already provided the rational numbers z_i.
- The secret key bits s_i are strictly 0 or 1.
- Our only job is to sum up numbers. Because $\vec{s}$ is sparse, only θ of them are non-zero!

Lemma (4.1: Elementary Symmetric Polynomials)

We use the classic "three-for-two" trick to reduce the sum. The i-th bit of the Hamming weight of a binary vector $\vec{\sigma}$ can be expressed as a polynomial of degree exactly 2^i using the 2^i-th elementary symmetric polynomial $e_{2^i}(\vec{\sigma})$.

Bounding the Degree

Because we only need $\lceil \log \theta \rceil + 3$ bits of precision after the binary point, the maximum degree required to compute the binary sum and rounding is strictly bounded by:

$$\text{Degree} \leq 64\theta \log^2 \theta$$

Bounding the Degree

Because we only need $\lceil \log \theta \rceil + 3$ bits of precision after the binary point, the maximum degree required to compute the binary sum and rounding is strictly bounded by:

$$\text{Degree} \leq 64\theta \log^2 \theta$$

Intuition

By forcing the evaluator to pre-compute the partial products z_i using the hint y_i, we squashed the massively deep integer division circuit down to a simple, sparse subset sum.

The Grand Finale: Bootstrapping Achieved

By setting $\theta = \lambda$, the degree of our squashed decryption polynomial becomes $\mathcal{O}(\lambda \log^2 \lambda)$.

The Grand Finale: Bootstrapping Achieved

By setting $\theta = \lambda$, the degree of our squashed decryption polynomial becomes $\mathcal{O}(\lambda \log^2 \lambda)$.

Theorem (6.2: Bootstrapping Achieved)

If we set our secret key length $\eta = \rho \cdot \Theta(\lambda \log^2 \lambda)$, the base SHE scheme's noise budget can comfortably handle polynomials of degree $\mathcal{O}(\lambda \log^2 \lambda)$.

The Grand Finale: Bootstrapping Achieved

By setting $\theta = \lambda$, the degree of our squashed decryption polynomial becomes $\mathcal{O}(\lambda \log^2 \lambda)$.

Theorem (6.2: Bootstrapping Achieved)

If we set our secret key length $\eta = \rho \cdot \Theta(\lambda \log^2 \lambda)$, the base SHE scheme's noise budget can comfortably handle polynomials of degree $\mathcal{O}(\lambda \log^2 \lambda)$.
Because the degree of the squashed decryption circuit is strictly less than the maximum degree of the permitted polynomials, we have:

$$\mathcal{D}_{\mathcal{E}} \subset \mathcal{C}_{\mathcal{E}}$$

Therefore, the scheme is bootstrappable.

The Grand Finale: Bootstrapping Achieved

By setting $\theta = \lambda$, the degree of our squashed decryption polynomial becomes $\mathcal{O}(\lambda \log^2 \lambda)$.

Theorem (6.2: Bootstrapping Achieved)

If we set our secret key length $\eta = \rho \cdot \Theta(\lambda \log^2 \lambda)$, the base SHE scheme's noise budget can comfortably handle polynomials of degree $\mathcal{O}(\lambda \log^2 \lambda)$.
Because the degree of the squashed decryption circuit is strictly less than the maximum degree of the permitted polynomials, we have:

$$\mathcal{D}_{\mathcal{E}} \subset \mathcal{C}_{\mathcal{E}}$$

Therefore, the scheme is bootstrappable.

We have achieved Fully Homomorphic Encryption using nothing but elementary modular arithmetic over the integers.

References

- Marten van Dijk, Craig Gentry, Shai Halevi, and Vinod Vaikuntanathan. *Fully Homomorphic Encryption over the Integers*. Eurocrypt, 2010.
- Craig Gentry. *A Fully Homomorphic Encryption Scheme*. PhD Thesis, Stanford University, 2009.
- J. Boyar, R. Peralta, and D. Pochuev. *On the multiplicative complexity of boolean functions over the basis ($\wedge$, $\oplus$, 1)*. Theor. Comput. Sci., 2000.